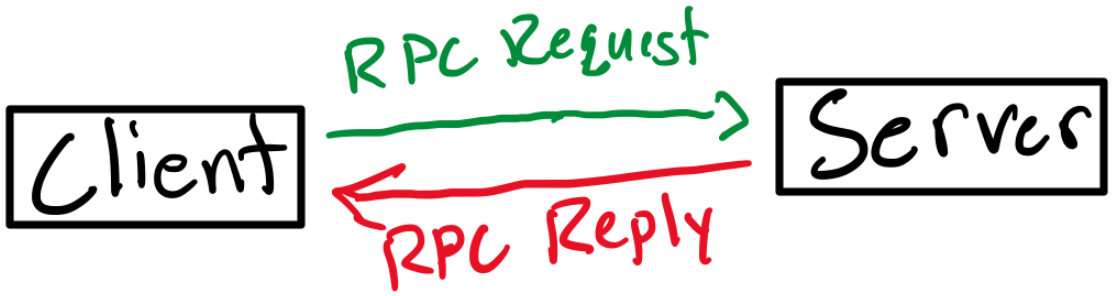
Introduction: The objective of this project is to design and implement a remote system monitoring application using Remote Procedure Calls (RPC). The system allows a client program to query a server running on a remote machine and retrieve system information using Remote Procedure Calls. The system follows a client–server architecture using Sun RPC for communication. The server runs on a host machine and provides system information. The client connects to the server and requests this information remotely. All communication occurs through RPC calls defined in an interface file.

Design:



Interface Design: The RPC interface is defined in the file date.x. The RPC program number are the RPC version number, the remote procedure, and the argument and return data types. The interface is processed using rpcgen, which automatically generates the client and server stubs.

RPC Procedure: The server provides a single RPC procedure that accepts a request from the client and returns system-related information. The main files are date.x, which shares the interface header, data_svc.c, which always contains the server-side RPC dispatcher, and data_clnt.c, which includes the client-side RPC stub.

Server responsibilities: The system operates using a client-server architecture and Sun RPC. The server registers itself with the RPC port mapper. It listens for incoming RPC requests, executes the requested procedure, and returns the result to the client.

Client responsibilities: The client is responsible for creating an RPC client handle, invoking the remote procedure on the server, and receiving and displaying the server's response

Build Process:

Generate RPC code: rpcgen date.x

Compile server: gcc -o server server.c date_svc.c

Compile client: gcc -o client client.c date_clnt.c

Start the server: ./server

Run the client: ./client localhost

Testing: I conducted a test by running both the server and the client on the same machine. First, I verified whether RPC registration was successful. After that, I confirmed whether the correct data was being sent back to the client. Lastly, I ensured there were no runtime errors during execution. The compiler warnings on macOS (The computer I am using) were checked and found to be fine and expected due to old RPC code.

Conclusion: This project demonstrates the success of using Sun RPC to create a remote system monitoring tool. The design puts the spotlight on the interaction between client and server, defining RPC interface and real-world difficulties of employing old-fashioned RPC systems on current platforms. At the end, I have a working system that is easy to move around and prepared for demonstration.